Lista de Exercícios — Herança em C#

Orientações

- Crie um projeto Console em C# para desenvolver os exercícios.
- Utilize classes para representar as entidades propostas.
- Aplique os conceitos de herança e orientação a objetos.
- Utilize `abstract`, `protected`, `base`, `virtual` e `override` quando necessário.
- Utilize `Console.WriteLine()` para apresentar os resultados.
- Organize o código em classes separadas.

Exercícios

1. Sistema de veículos

Uma empresa precisa representar diferentes tipos de veículos.

Crie uma classe base `Veiculo`, contendo as propriedades `Marca` e `Modelo` e um método `ExibirInformacoes()`.

Em seguida, crie duas classes derivadas:

- `Carro`: deve possuir a propriedade `QuantidadePortas`.
- `Motocicleta`: deve possuir a propriedade `Cilindradas`.

Crie objetos das duas classes e apresente suas informações.

2. Sistema de produtos digitais

Uma plataforma comercializa diferentes tipos de produtos digitais.

Crie uma classe abstrata `ProdutoDigital`, contendo as propriedades `Nome` e `Preco`.

Implemente um construtor para inicializar essas propriedades.

Em seguida, crie duas classes derivadas:

- `Ebook`: deve possuir a propriedade `QuantidadePaginas`.

- `CursoOnline`: deve possuir a propriedade `CargaHoraria`.

Utilize `base` para chamar o construtor da classe base.

3. Sistema de jogos

Uma desenvolvedora precisa representar diferentes tipos de jogos.

Crie uma classe abstrata `Jogo`, contendo:

- Propriedade `Nome`.
- Campo protegido `Plataforma`.
- Método para apresentar as informações do jogo.

Crie duas classes derivadas:

- `JogoDigital`: deve apresentar informações sobre o formato digital.
- `JogoFisico`: deve apresentar informações sobre o formato físico.

Utilize construtores para inicializar os dados e permita que as classes derivadas acessem o campo `Plataforma`.

4. Sistema de notificações

Uma aplicação precisa enviar notificações por diferentes canais.

Crie uma classe base `Notificacao` com um método virtual `Enviar()` que apresente uma mensagem genérica.

Crie três classes derivadas:

- `NotificacaoEmail`.
- `NotificacaoSMS`.
- `NotificacaoPush`.

Cada classe deverá sobrescrever o método `Enviar()`, apresentando uma mensagem específica de acordo com o canal utilizado.

5. Sistema de cálculo de frete

Uma loja virtual precisa calcular o frete de acordo com a modalidade de entrega.

Crie uma classe abstrata `Entrega` com:

- Propriedade `Distancia`.

- Método abstrato `CalcularFrete()`.

Crie três classes derivadas:

- `EntregaExpressa`: calcula o frete considerando uma taxa adicional.
- `EntregaEconomica`: calcula o frete utilizando uma tarifa reduzida.
- `RetiradaLocal`: não possui custo de frete.

Implemente o cálculo em cada classe utilizando `override`.

Em seguida, crie uma lista do tipo `Entrega`, adicione objetos das três modalidades e utilize `foreach` para apresentar o valor do frete de cada uma.

6. Sistema de documentos

Desenvolva um pequeno sistema de gerenciamento de documentos utilizando interfaces em C#.

1. Crie uma interface `IImprimivel` com o método `Imprimir()`.
2. Crie uma interface `IExportavel` com o método `Exportar()`.
3. Crie uma classe `Relatorio` que implemente as duas interfaces.
4. Crie uma classe `Contrato` que implemente apenas a interface `IImprimivel`.
5. Implemente os métodos de acordo com o comportamento de cada classe, exibindo mensagens no console.
6. No programa principal, instancie os objetos e execute os métodos disponíveis.